

Reinforcement Learning 2

Anders Lyhne Christensen

(slides based on Sutton and Barto's slides and book)

Last time

- *Reinforcement learning*: an agent learns through interactions with an environment.
- *k*-armed bandit problem: The true distribution of rewards is not known a priori, but must be learned.



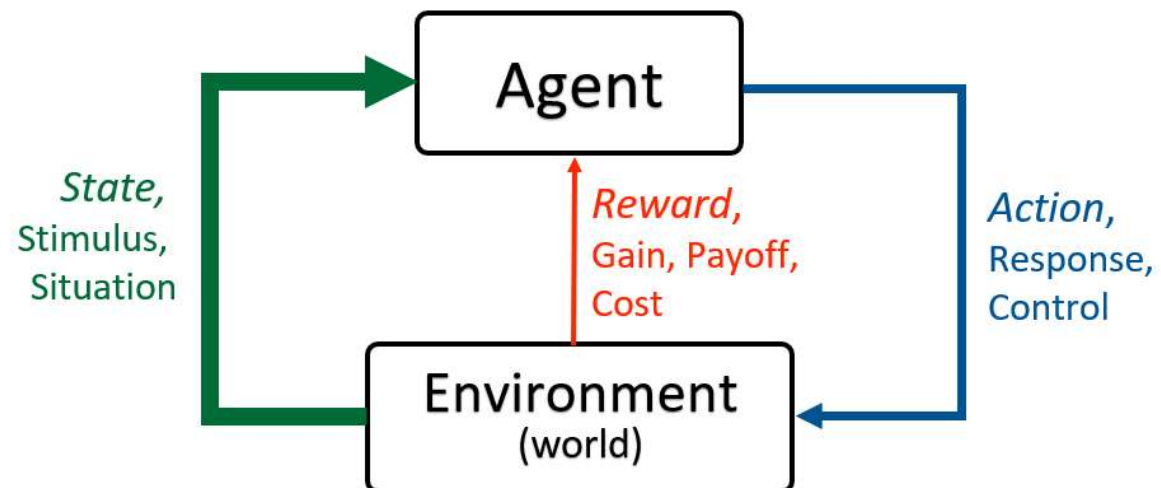
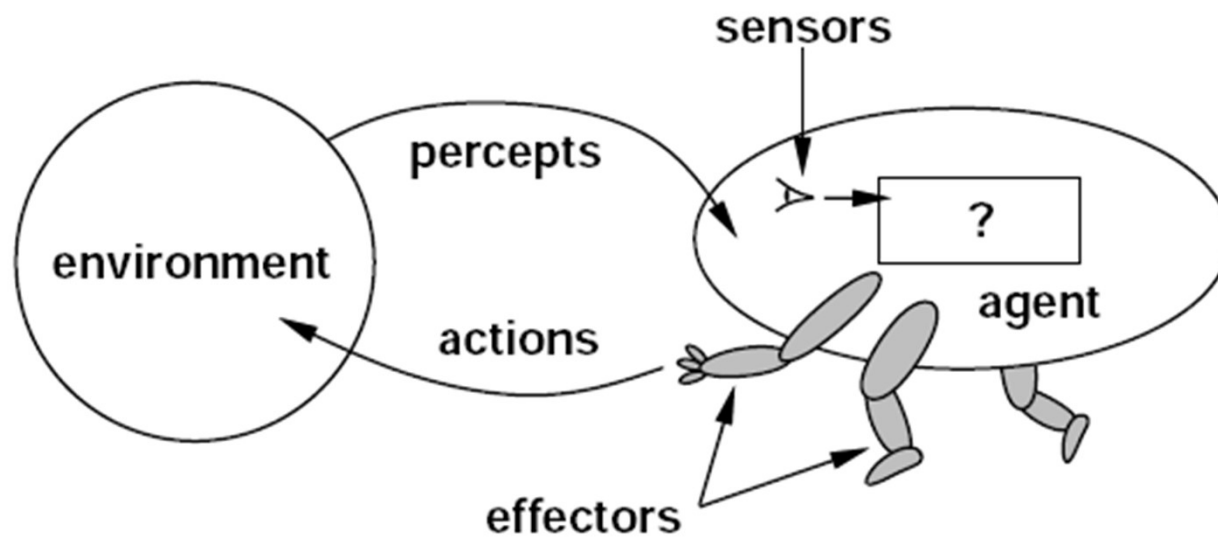
$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a]$$

$$Q_t(a) \approx q_*(a), \quad \forall a$$

Last time

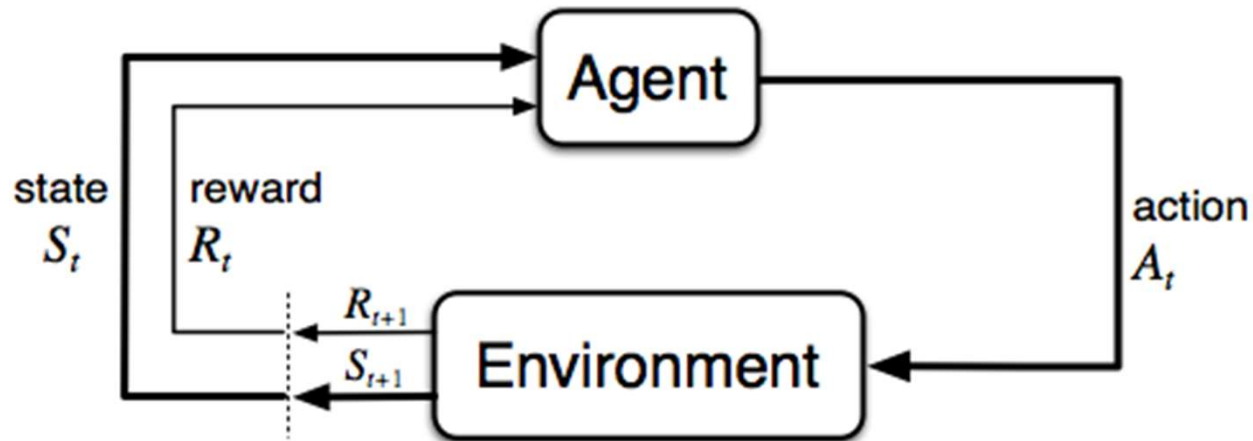
We discussed:

- UniformBandit
- Exploration vs. exploitation tradeoff
- ε -Greedy
- How to reduce the bias of initial estimates $Q_1(a)$, namely a fixed step-size α .
- Optimistic initialization ($Q_1(a) = 5$)
- Upper confidence bound: $A_t \doteq \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$
- Gradient bandit algorithm: preferences are used instead of estimates of q_*



Markov Decision Processes

The Agent-Environment Interface



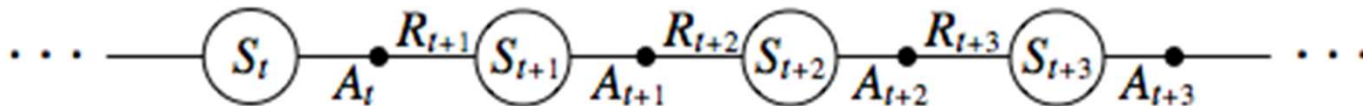
Actions at discrete timesteps: $t = 0, 1, 2, \dots, k$

Agent observes state at step t : $S_t \in \mathcal{S}$

produces action at step t : $A_t \in \mathcal{A}(s_t)$

gets reward: $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$

and next state: $S_{t+1} \in \mathcal{S}^+$



Markov Decision Processes

If a reinforcement learning task has the *Markov property*, it is a **Markov Decision Process (MDP)**.

A stochastic process has the *Markov property* if the conditional probability distribution of future states of the process (conditional on both past and present states) **depends only upon the present state**, not on the sequence of events that preceded it. A process with this property is called a *Markov process*.

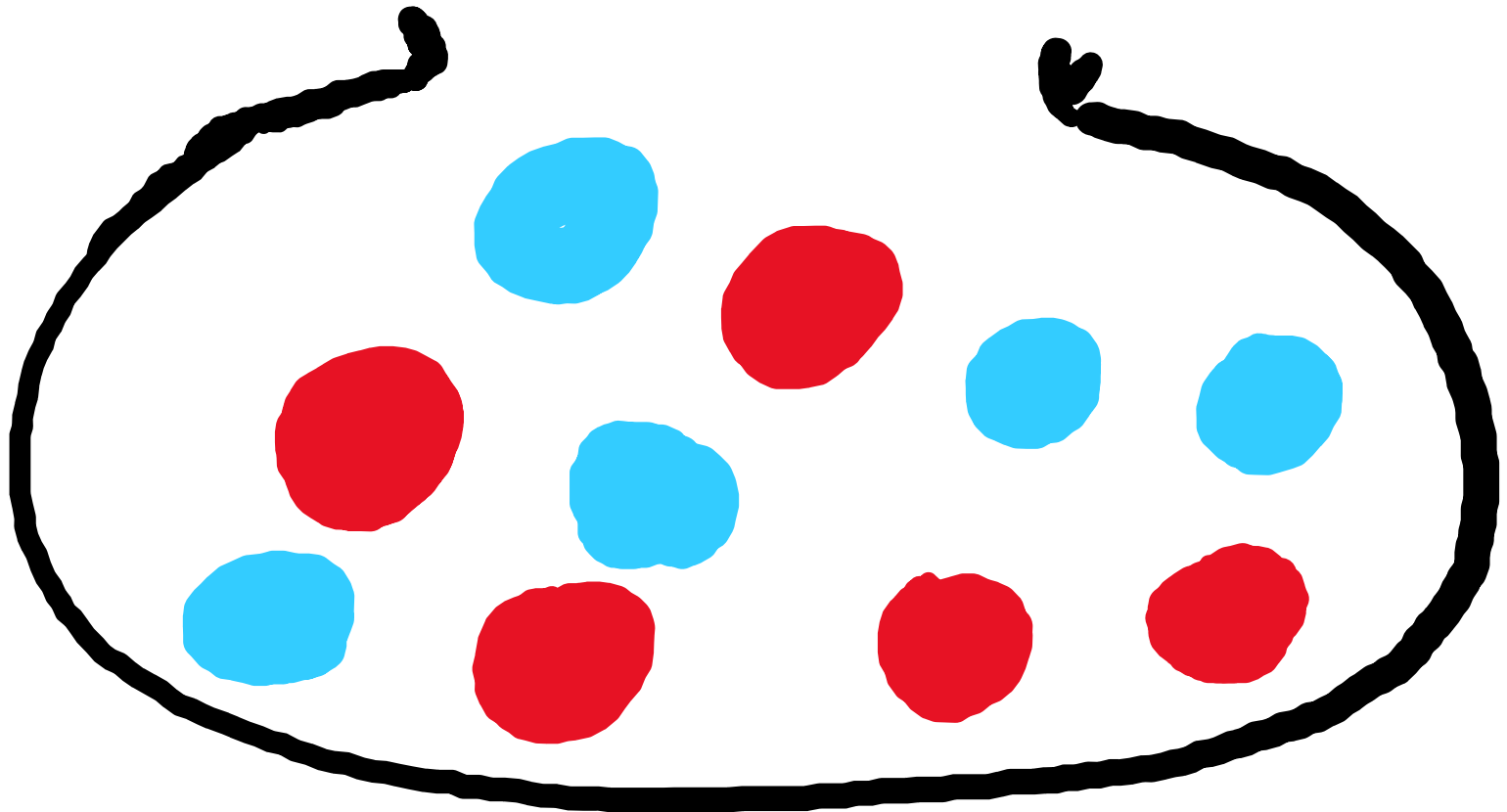
From Wikipedia.

$$\Pr\{R_{t+1} = r, S_{t+1} = s' \mid S_0, A_0, R_1, \dots, S_{t-1}, A_{t-1}, R_t, S_t, A_t\} =$$

$$p(s', r | s, a) = \Pr\{R_{t+1} = r, S_{t+1} = s' \mid S_t, A_t\}$$

Example

Successively draw balls from a bowl. Probability of drawing a red vs. a blue?

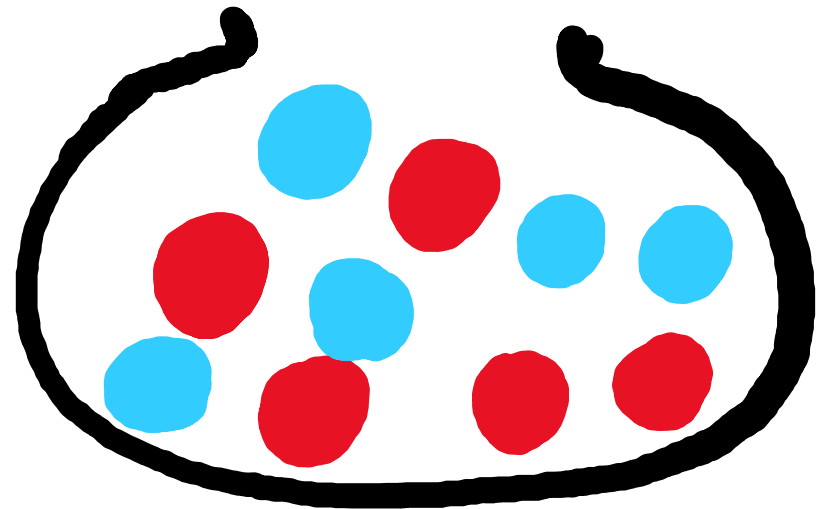


Example

State = *nil*

Without replacement: not a Markov process

With replacement: a Markov process



Note that the Markov property is best views as restriction ***not* on the decision process, but on the state**.

Markov Decision Processes

If state and action sets are finite, it is a **finite MDP**. To define a finite MDP, you need to give:

- State and action sets:

$$\mathcal{S} \text{ and } \mathcal{A}$$

- one-step “dynamics”:

$$p(s', r | s, a) = \mathbf{Pr}\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$$

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s)$$

Examples

$$p(s', r | s, a) = \mathbf{Pr}\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$$

States = {a, ..., i}

Actions = {U, D, L, R, UL, DL, UR, DR }

Reward = -1 if collide with a wall,
0 otherwise

$$p(d, -1 \mid e, L) =$$

$$p(d, 0 \mid e, L) =$$

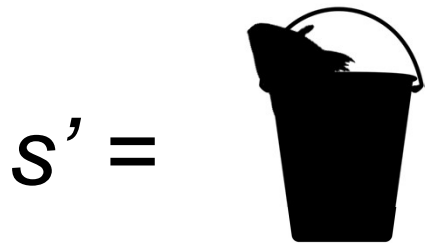
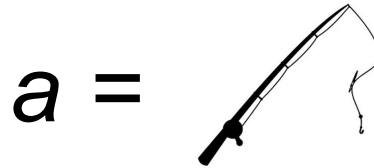
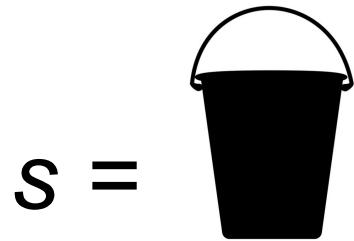
$$p(h, 0 \mid e, L) =$$

$$p(i, -1 \mid i, R) =$$

a	b	c
d	 e	f
g	h	i

Examples

$$p(s', r | s, a) = \mathbf{Pr}\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$$



$r = \textit{nothing}$

Markov Decision Processes

From the dynamics:

$$p(s', r | s, a) = \mathbf{Pr}\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$$

we can easily compute other convenient quantities, such as the *state-transition probabilities*:

$$p(s' | s, a) \doteq \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a)$$

and the expected reward in state s of taking action a :

$$r(s, a) \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r | s, a)$$

and expected reward in state s of taking action a when ending in s' :

$$r(s, a, s') \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r | s, a)}{p(s' | s, a)}$$

Abstraction Enables Generality

- *Time steps* need not refer to fixed intervals of real time.
- *Actions* can be low level (e.g., voltages to motors), or high level (e.g., accept a job offer), “mental” (e.g., shift in focus of attention), etc.
- *States* can low-level “sensations”, or they can be abstract, symbolic, based on memory, or subjective (e.g., the state of being “surprised” or “lost”).
- An RL agent is not necessarily a *whole* animal or robot.
- *Reward* computation is in the agent’s environment because the agent cannot change it arbitrarily.
- The environment is not necessarily unknown to the agent, only incompletely controllable.

An Example Finite MDP

Recycling Robot

- At each step, robot has to decide whether it should (1) actively search for a can, (2) wait for someone to bring it a can, or (3) go to home base and recharge.
- Searching is better but runs down the battery; if runs out of power while searching, has to be rescued (which is bad).
- Decisions made on basis of current energy level: *high, low*.
- Reward = *number of cans collected*



Recycling Robot MDP

$$\mathcal{S} = \{\text{high}, \text{low}\}$$

$$\mathcal{A}(\text{high}) = \{\text{search}, \text{wait}\}$$

$$\mathcal{A}(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$$

α = probability that searching in **high** leaves energy level **high**

r_{search} = expected no. of cans while searching

r_{wait} = expected no. of cans while waiting

$$r_{\text{search}} > r_{\text{wait}}$$

β = probability that searching in **low** leaves energy level **low**

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

Recycling Robot MDP

$$\mathcal{S} = \{\text{high}, \text{low}\}$$

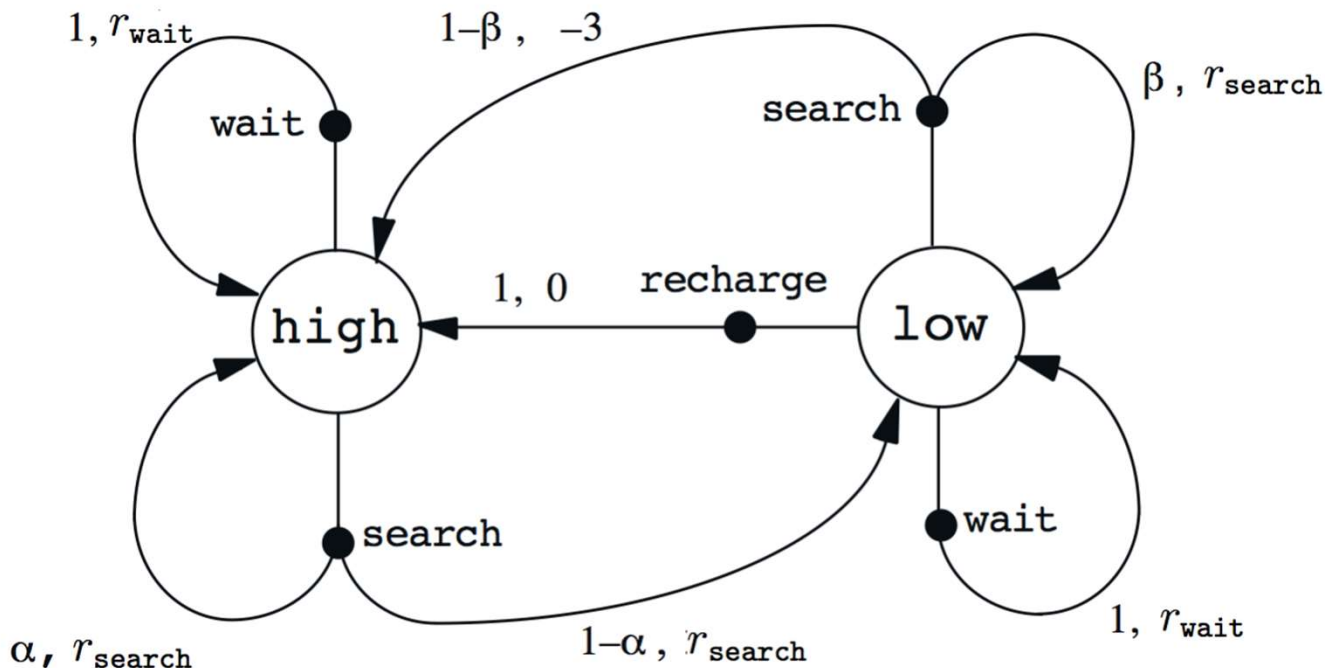
$$\mathcal{A}(\text{high}) = \{\text{search}, \text{wait}\}$$

$$\mathcal{A}(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$$

r_{search} = expected no. of cans while searching

r_{wait} = expected no. of cans while waiting

$$r_{\text{search}} > r_{\text{wait}}$$



α = probability that searching in **high** leaves energy level **high**

β = probability that searching in **low** leaves energy level **low**

The Meaning of Life (goals, rewards, and returns)

A scalar value?

Goals and Rewards

- Is a scalar reward signal an adequate notion of a goal?—maybe not, but it is surprisingly flexible.
- A goal should specify **what** we want to achieve, not **how** we want to achieve it.
- A goal must be outside the agent's direct control—thus outside the agent.
- The agent must be able to measure success:
 - explicitly;
 - frequently during its lifespan (otherwise, it can be challenging to learn).

Rewards and returns

- The objective in RL is to maximize long-term future reward
- That is, to choose A_t so as to maximize $R_{t+1}, R_{t+2}, R_{t+3}, \dots$
- But what exactly should be maximized?

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

Episodic vs. continuing tasks

An episodic task is one that ends at some finite time T :

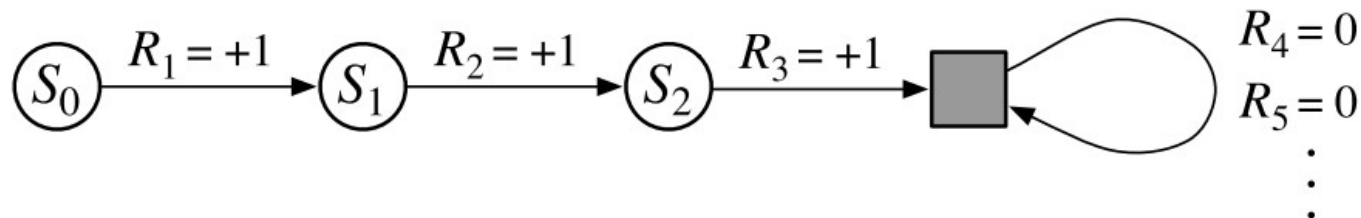
$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T$$

(ends when the process ends in the terminal state at T)

A continuing task is one that goes on forever:

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \cdots$$

To make an episodic task into a continuing task, we introduce a special *absorbing state*:



The Agent Learns a Policy

Policy at step t = π_t =

a mapping from states to action probabilities

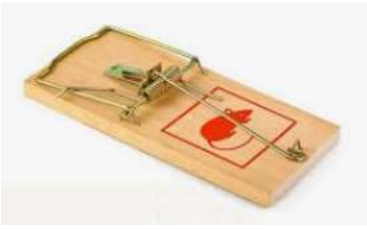
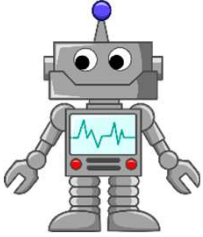
$\pi_t(a | s)$ = probability that $A_t = a$ when $S_t = s$

Special case - *deterministic policies*:

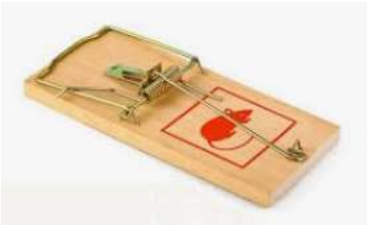
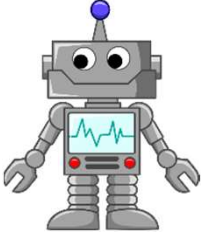
$\pi_t(s)$ = the action taken with prob=1 when $S_t = s$

- Reinforcement learning methods specify how the agent *changes its policy as a result of experience*.
- Roughly, the agent's goal is to get as much reward as it can over the long run.

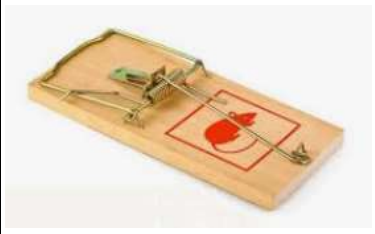
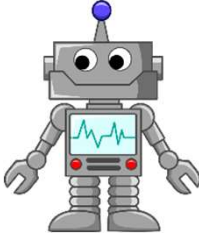
Example

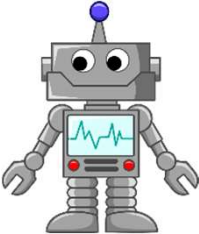
Example

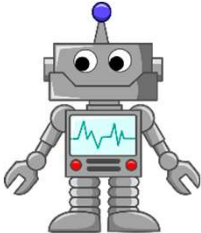
Example

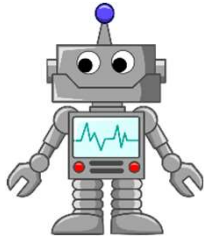
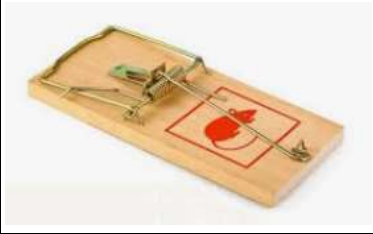
Example

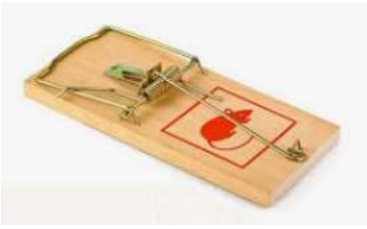
Example

Example

Example

Rewards and returns

- The discounted *return* at time t :

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \quad \begin{array}{l} \text{the } \textit{discount rate} \\ \gamma \in [0, 1) \end{array}$$

γ	Reward sequence	Return
0.5(or any)	1 0 0 0...	
0.5	0 0 2 0 0 0...	
0.9	0 0 2 0 0 0...	
0.5	-1 2 6 3 2 0 0 0...	

Rewards and returns

- The discounted *return* at time t :

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \quad \begin{array}{l} \text{the } \textit{discount rate} \\ \gamma \in [0, 1) \end{array}$$

γ	Reward sequence	Return
0.5(or any)	1 0 0 0...	1
0.5	0 0 2 0 0 0...	
0.9	0 0 2 0 0 0...	
0.5	-1 2 6 3 2 0 0 0...	

Rewards and returns

- The discounted *return* at time t :

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \quad \begin{array}{l} \text{the } \textit{discount rate} \\ \gamma \in [0, 1) \end{array}$$

γ	Reward sequence	Return
0.5(or any)	1 0 0 0...	1
0.5	0 0 2 0 0 0...	0.5
0.9	0 0 2 0 0 0...	
0.5	-1 2 6 3 2 0 0 0...	

Rewards and returns

- The discounted *return* at time t :

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \quad \begin{array}{l} \text{the } \textit{discount rate} \\ \gamma \in [0, 1) \end{array}$$

γ	Reward sequence	Return
0.5(or any)	1 0 0 0...	1
0.5	0 0 2 0 0 0...	0.5
0.9	0 0 2 0 0 0...	1.62
0.5	-1 2 6 3 2 0 0 0...	

Rewards and returns

- The discounted *return* at time t :

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \quad \begin{array}{l} \text{the } \textit{discount rate} \\ \gamma \in [0, 1) \end{array}$$

γ	Reward sequence	Return
0.5(or any)	1 0 0 0...	1
0.5	0 0 2 0 0 0...	0.5
0.9	0 0 2 0 0 0...	1.62
0.5	-1 2 6 3 2 0 0 0...	2

Discounted return G_t is finite if $\gamma < 1$

- The infinite sum converges:

$$\sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}$$

- If all rewards are finite, so is the discounted return:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$

Four value functions

	state values	action values
prediction	v_{π}	q_{π}
Optimal (for control)	v_{*}	q_{*}

- All theoretical objects, mathematical ideals (expected values)
- Distinct from their *estimates*: $V_t(s)$ $Q_t(s, a)$

Values are *expected* returns

- The value of a state, given a policy:

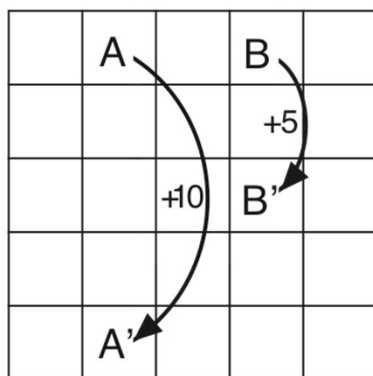
$$v_{\pi}(s) = \mathbb{E}\{G_t \mid S_t = s, A_{t:\infty} \sim \pi\} \quad v_{\pi} : \mathcal{S} \rightarrow \mathbb{R}$$

- The value of a state-action pair, given a policy:

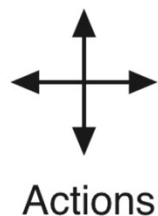
$$q_{\pi}(s, a) = \mathbb{E}\{G_t \mid S_t = s, A_t = a, A_{t+1:\infty} \sim \pi\} \quad q_{\pi} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

Gridworld

- Actions: north, south, east, west; deterministic.
- If action would take agent off the grid: no move but reward = -1
- Other actions produce reward = 0, except actions that move agent out of special states A and B as shown.



(a)



3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

(b)

State-value function
for equiprobable
random policy;
 $\gamma = 0.9$

Values are *expected* returns

- The value of a state, given a policy:

$$v_{\pi}(s) = \mathbb{E}\{G_t \mid S_t = s, A_{t:\infty} \sim \pi\} \quad v_{\pi} : \mathcal{S} \rightarrow \mathbb{R}$$

- The value of a state-action pair, given a policy:

$$q_{\pi}(s, a) = \mathbb{E}\{G_t \mid S_t = s, A_t = a, A_{t+1:\infty} \sim \pi\} \quad q_{\pi} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

Values are *expected* returns

- The value of a state, given a policy:

$$v_{\pi}(s) = \mathbb{E}\{G_t \mid S_t = s, A_{t:\infty} \sim \pi\} \quad v_{\pi} : \mathcal{S} \rightarrow \mathbb{R}$$

- The value of a state-action pair, given a policy:

$$q_{\pi}(s, a) = \mathbb{E}\{G_t \mid S_t = s, A_t = a, A_{t+1:\infty} \sim \pi\} \quad q_{\pi} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

- The optimal value of a state:

$$v_{*}(s) = \max_{\pi} v_{\pi}(s) \quad v_{*} : \mathcal{S} \rightarrow \mathbb{R}$$

- The optimal value of a state-action pair:

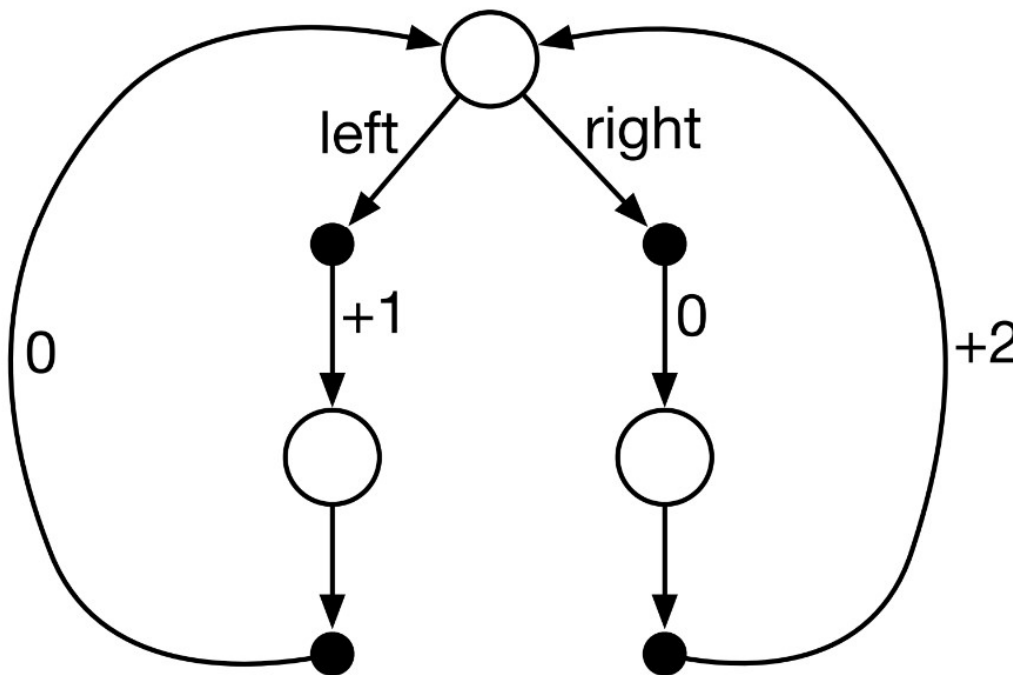
$$q_{*}(s, a) = \max_{\pi} q_{\pi}(s, a) \quad q_{*} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

- Optimal policy: π_{*} is an optimal policy if and only if

$$\pi_{*}(a|s) > 0 \text{ only where } q_{*}(s, a) = \max_b q_{*}(s, b) \quad \forall s \in \mathcal{S}$$

- in other words, π_{*} is optimal iff it is *greedy* wrt q_{*}

Optimal policy example



What is an optimal policy for:

$$\gamma = 0$$

$$\gamma = 0.9$$

$$\gamma = 0.5$$

Bellman Equation for a Policy π

The basic idea:

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

So:

$$\begin{aligned} v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \end{aligned}$$

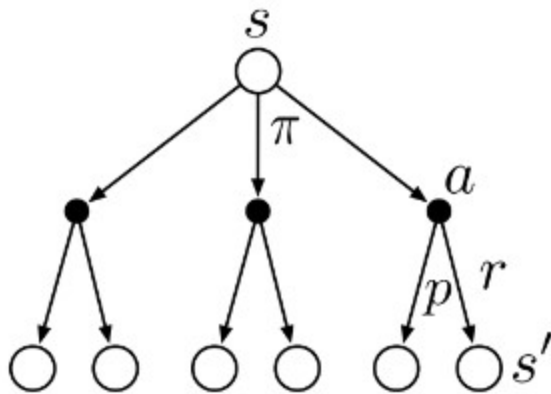
Or, without the expectation operator:

$$= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')], \quad \text{for all } s \in \mathcal{S}$$

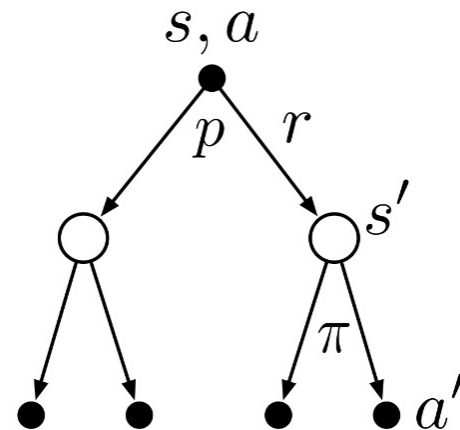
Backup diagrams

$$v_{\pi}(s) \doteq \sum \pi(a|s) \sum p(s', r | s, a) \left[r + \gamma v_{\pi}(s') \right], \quad \text{for all } s \in \mathcal{S}$$

Backup diagrams:



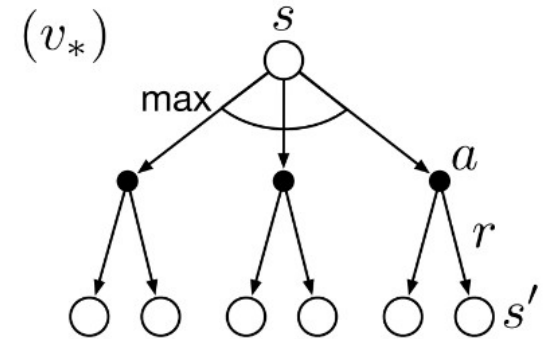
Backup diagram for v_{π}



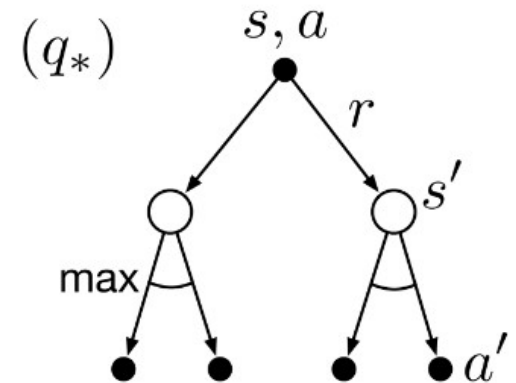
Backup diagram for q_{π}

Bellman Optimality Equation for v_* and q_*

$$\begin{aligned}
 v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\
 &= \max_a \mathbb{E}_{\pi_*}[G_t \mid S_t = s, A_t = a] \\
 &= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\
 &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\
 &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')].
 \end{aligned}$$



$$\begin{aligned}
 q_*(s, a) &= \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a] \\
 &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma \max_{a'} q_*(s', a')].
 \end{aligned}$$



Four value functions

	state values	action values
prediction	v_{π}	q_{π}
optimal (for control)	v_{*}	q_{*}

- All theoretical objects, mathematical ideals (expected values)
- Distinct from their *estimates*: $V_t(s)$ $Q_t(s, a)$

4 x Value Functions =
4 x Bellman Equations

Solving the Bellman Optimality Equation

- Finding an optimal policy by solving the Bellman Optimality Equation requires the following:
 - accurate knowledge of environment dynamics;
 - we have enough space and time to do the computation;
 - the Markov Property.
- How much space and time do we need?
 - polynomial in number of states (via dynamic programming methods),
 - BUT, number of states is often huge (e.g., backgammon has about 10^{20} states).
- We usually have to settle for approximations.
- Many RL methods can be understood as approximately solving the Bellman Optimality Equation.

Iterative Methods

$$v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_k \rightarrow v_{k+1} \rightarrow \cdots \rightarrow v_\pi$$

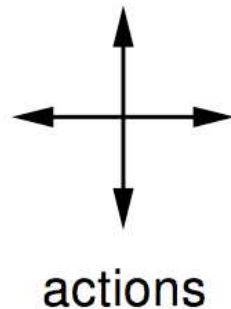
a “sweep” 

A sweep consists of applying a **backup operation** to each state.

A **full policy-evaluation backup**:

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_k(s') \right] \quad \forall s \in \mathcal{S}$$

A Small Gridworld



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R = -1$
on all transitions

$$\gamma = 1$$

- An undiscounted episodic task
- Nonterminal states: 1, 2, . . . , 14;
- One terminal state (shown twice as shaded squares)
- Actions that would take agent off the grid leave state unchanged
- Reward is -1 until the terminal state is reached

Iterative Policy Eval for the Small Gridworld

V_k for the
Random Policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

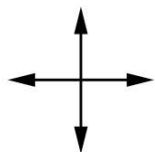
$k = 2$

$k = 3$

$k = 10$

$k = \infty$

π : equiprobably random action choices



actions

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

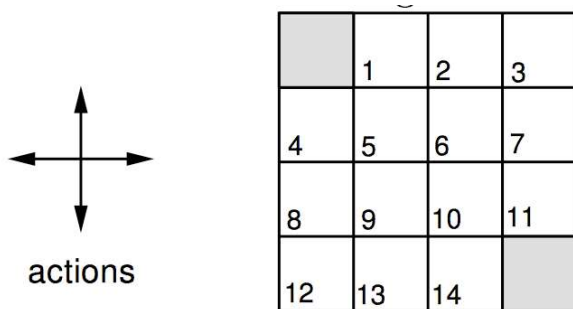
$R = -1$
on all transitions

$\gamma = 1$

- An undiscounted episodic task
- Nonterminal states: 1, 2, ..., 14;
- One terminal state (shown twice as shaded squares)
- Actions that would take agent off the grid leave state unchanged
- Reward is -1 until the terminal state is reached

Iterative Policy Eval for the Small Gridworld

π : equiprobably random action choices



$R = -1$
on all transitions

$\gamma = 1$

V_k for the
Random Policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

$k = 3$

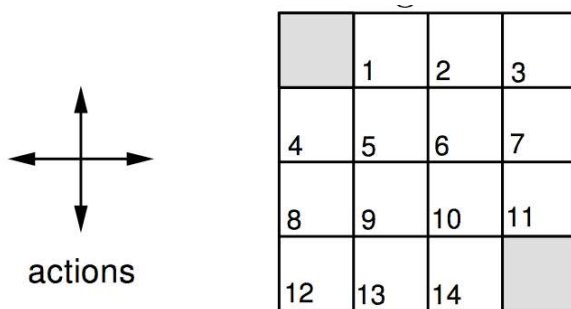
$k = 10$

$k = \infty$

- An undiscounted episodic task
- Nonterminal states: 1, 2, ..., 14;
- One terminal state (shown twice as shaded squares)
- Actions that would take agent off the grid leave state unchanged
- Reward is -1 until the terminal state is reached

Iterative Policy Eval for the Small Gridworld

π : equiprobably random action choices



$R = -1$
on all transitions

$\gamma = 1$

V_k for the
Random Policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 3$

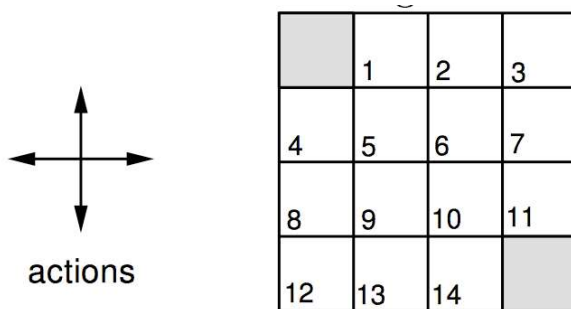
$k = 10$

$k = \infty$

- An undiscounted episodic task
- Nonterminal states: 1, 2, ..., 14;
- One terminal state (shown twice as shaded squares)
- Actions that would take agent off the grid leave state unchanged
- Reward is -1 until the terminal state is reached

Iterative Policy Eval for the Small Gridworld

π : equiprobably random action choices



$R = -1$
on all transitions

$\gamma = 1$

- An undiscounted episodic task
- Nonterminal states: 1, 2, ..., 14;
- One terminal state (shown twice as shaded squares)
- Actions that would take agent off the grid leave state unchanged
- Reward is -1 until the terminal state is reached

V_k for the
Random Policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 3$

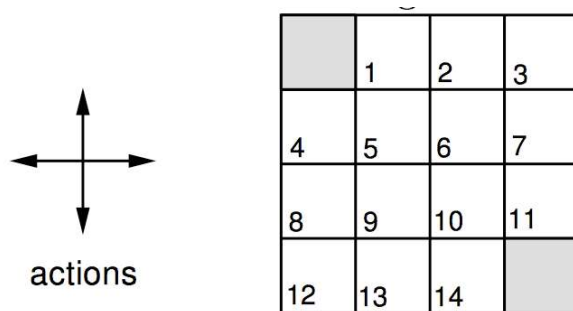
0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 10$

$k = \infty$

Iterative Policy Eval for the Small Gridworld

π : equiprobably random action choices



$R = -1$
on all transitions

$\gamma = 1$

- An undiscounted episodic task
- Nonterminal states: 1, 2, ..., 14;
- One terminal state (shown twice as shaded squares)
- Actions that would take agent off the grid leave state unchanged
- Reward is -1 until the terminal state is reached

V_k for the
Random Policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

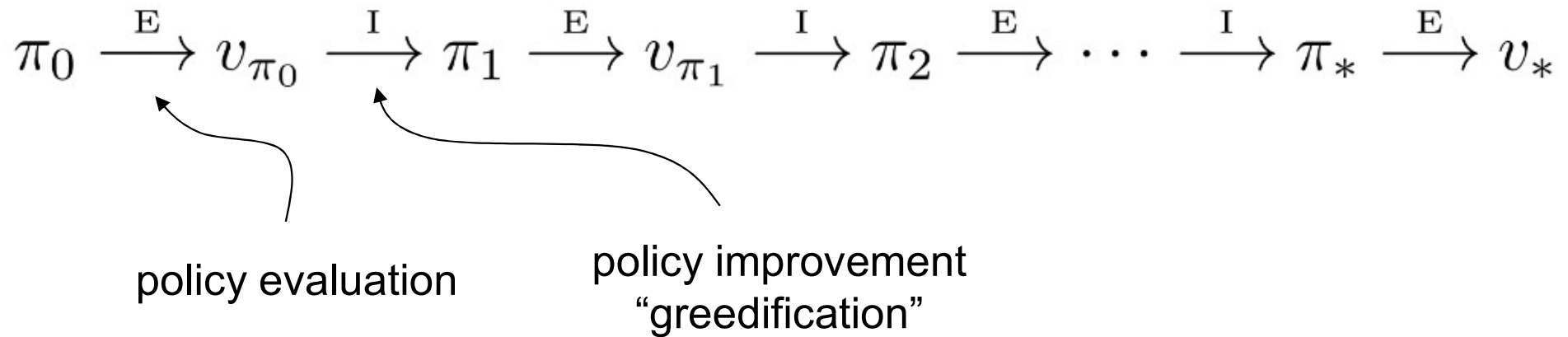
$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

Policy Iteration

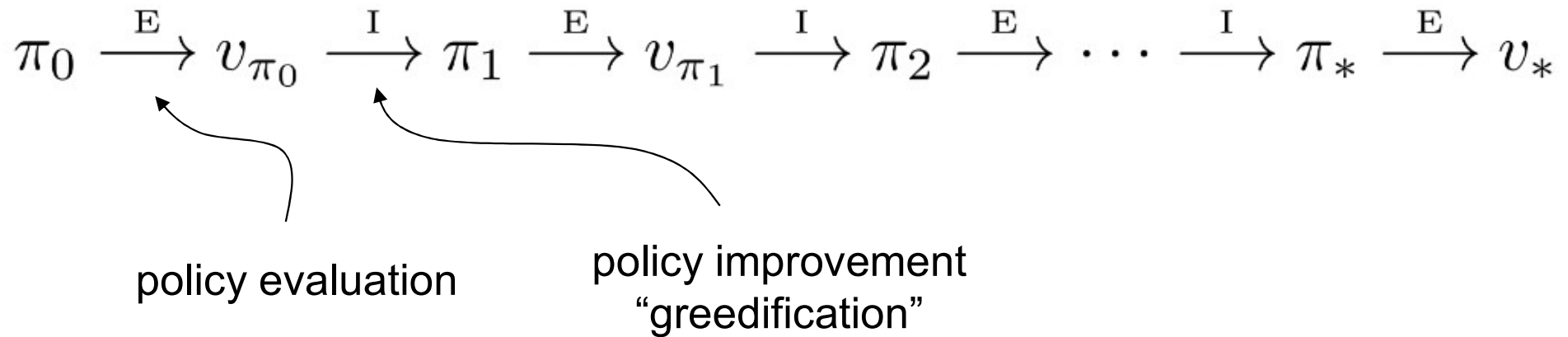


Policy Iteration

- When should a policy be changed?

$$q_{\pi}(s, a) > v_{\pi}(s)$$

Policy Iteration



Value iteration

- **Policy iteration** may be slow since each iteration requires a policy evaluation:

$$\begin{aligned}v_{k+1}(s) &\doteq \mathbb{E}_{\pi}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\&= \sum_a \pi(a|s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')]\end{aligned}$$

- **Value iteration:** If we combine policy evaluation with policy improvement, we can find the optimal policy faster:

$$\begin{aligned}v_{k+1}(s) &\doteq \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] \\&= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')],\end{aligned}$$

Value Iteration algorithm

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that
$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

Summary

- Finite Markov decision processes. Dynamics defined by:

$$p(s', r | s, a) = \mathbf{Pr}\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$$

- Discounted return:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$

- Policy:

$$\pi_t(a | s)$$

- Value functions v_π, q_π, v_*, q_*
- Bellman equations – exploiting the recursive structure of value functions.
- Iterative methods, policy iteration and value iteration.